

- [exp3: AlexNet TensorRT INT8 推理实验](#)
  - [运行环境](#)
  - [整体过程](#)
    - [INT8 量化和校准的原理补充](#)
    - [其它文件/文件夹在实验里的作用](#)
  - [结果和分析](#)
    - [单图推理](#)
    - [CIFAR-10 测试集](#)

## exp3: AlexNet TensorRT INT8 推理实验

(TensorRT / INT8 模型部署入门)

这个实验的主任务不是重新训练模型，而是使用 exp2 已经训练好的 FP32 AlexNet，通过 `torch2trt` 构建 TensorRT INT8 engine，然后比较 PyTorch FP32 与 TensorRT INT8 的单图预测结果、平均推理延迟以及完整 CIFAR-10 测试集准确率。

CIFAR-10 每张图片是 32x32 的 RGB 图片，共有 10 个类别：飞机、汽车、鸟、猫、鹿、狗、青蛙、马、船、卡车。

### 运行环境

运行前需要准备：

- 支持 CUDA/INT8 的 NVIDIA GPU；
- `~/pyenv` 虚拟环境及其中互相兼容的 PyTorch、torchvision、TensorRT、torch2trt 和 Pillow；
- 当前机器使用 CUDA 11.8 时，建议选择支持 CUDA 11.8 的 TensorRT 8.6.1 GA，而不是当前默认面向 CUDA 12/13 的新版本；

### 整体过程

- Python 启动 `int8_infer.py` 时，会先执行文件顶部的所有导入。其中：

```
from data import QDataset, testloader
```

会导入 `exp3/data.py`：

- 首先定义 CIFAR-10 测试集预处理 `test_transforms`：

- `ToTensor()`：把图片转换为 PyTorch 浮点张量，形状为 `[C, H, W]`，并把像素值从 `[0, 255]` 缩放到 `[0, 1]`；
- `Normalize(mean, std)`：对 RGB 三个通道分别执行  $(value - mean) / std$ ；
- 测试集不使用随机裁剪、翻转等随机增强，因为测试时需要让同一张图片每次得到稳定、可比较的结果。
- 根据自身位置找到 `exp1/data/`，并创建 CIFAR-10 测试集 `testset`，每次取出测试图片时执行 `test_transforms`
- 再用 `testset` 创建 `testloader`
  - `batch_size=1`：每次取一张测试图片和一个 label，与 TensorRT engine 的 `max_batch_size=1` 一致；
  - `shuffle=False`：不打乱测试集顺序；
  - `num_workers=4`：使用 4 个子进程读取和预处理数据。
- 最后定义 `QDataset` 校准数据集类：
  - `__init__()` 用 `glob("./data/q/*.jpg")` 查找 `exp3` 的校准图片，保存图片路径、图片数量和外部传入的 `transform`；
  - `__len__()` 返回 `self.len`，即校准图片数量；
  - `__getitem__(index)` 读取并预处理一张校准图片，添加 `batch` 维度后移到 GPU，最终按 `torch2trt` 的单输入格式返回 `[input_tensor]`。校准只需要图片，不需要 label。
- `data.py` 导入完成后，`int8_infer.py` 继续执行其它导入。为了明确使用 `exp2` 的模型结构，代码根据自身位置找到 `exp2`，再把它放到 Python 模块搜索路径最前面，让导入的 `models` 明确指向 `exp2/models/`；
  - 当模型名是 `alexnet` 时，最终会调用 `exp2/models/alexnet.py` 中的 `AlexNet()`。
- `int8_infer.py` 主流程：
  - 通过 `argparse` 读取 `--gpu` 和 `--model`：
    - `args.gpu=True` 且 `torch.cuda.is_available()` 为真时，启用 CuDNN 和 CuDNN benchmark，并执行 `model.cuda()`；
      - CuDNN benchmark 会针对固定的 `[1, 3, 32, 32]` 输入选择较快的卷积实现；
      - 如果没有传 `--gpu` 或机器没有可用 CUDA，程序打印提示并退出，因为 TensorRT 推理依赖 NVIDIA GPU。
  - `model_factory(model_name)` 创建的只是网络结构，接下来还要加载 `exp2` 训练出的 FP32 参数。本实验固定读取 `exp3/weights/alexnet/weights.130.83.050.pt`
    - `.pt` 被仓库根目录 `.gitignore` 中的 `*.pt` 忽略，因此运行前需要在目标机器自行放入权重。
  - 定义校准图片和单张测试图片共用的 `cali_augmentation`：
    - `Resize((32, 32), interpolation=Image.BICUBIC)`：用双三次插值缩放到 `32x32`；
    - `ToTensor()`：转成 `[3, 32, 32]` 浮点张量；
    - `Normalize(mean, std)`：使用与 `exp2` 训练/测试一致的 CIFAR-10 均值和标准差；

- 校准图片与正式推理图片必须使用相同预处理，否则校准时观察到的 activation values 分布可能不适合正式输入。
- 创建校准集对象 `cali_cifar10 = QDataset(transform=cali_augmentation)`。此时只是准备校准数据；真正使用校准集发生在后面的 `torch2trt()` 调用中。
- `model.eval()` 把 FP32 模型切换到评估模式：
  - BatchNorm 使用训练阶段保存的统计量；
  - Dropout 停止随机丢弃神经元；
  - `eval()` 不等于关闭梯度，实际推理阶段还会另外使用 `torch.no_grad()`
- 接下来构建 TensorRT INT8 engine：

```
model_trt_int8 = torch2trt(model, [x], fp16_mode=False, int8_mode=True,
    max_batch_size=1, int8_calib_dataset=cali_cifar10,
    int8_calib_algorithm=tensorrt.CalibrationAlgoType.ENTROPY_CALIBRATION_2,
)
```

- `model`：已经加载 FP32 权重并切换到 eval 模式的 AlexNet；
- `[x]`：AlexNet 的一个示例输入，模型输入必须放在列表中；
- `fp16_mode=False`：本实验不主动启用 FP16；
- `int8_mode=True`：要求 TensorRT 为支持的层启用 INT8；
- `max_batch_size=1`：engine 一次最多处理一张图片；
- `int8_calib_dataset=cali_cifar10`：把 data/q/ 校准图片交给 TensorRT；
- `ENTROPY_CALIBRATION_2`：使用熵校准算法选择 activation values 的 INT8 动态范围；
- 没有 INT8 实现的算子可能回退到其它精度，因此启用 INT8 不代表 engine 中每个算子一定执行 INT8。
- engine 构建完成后，读取干净单图 `test.jpeg`，执行相同的 `cali_augmentation`，再通过 `unsqueeze_(0)` 添加 batch 维度并用 `.cuda()` 移到 GPU，最终得到 `[1, 3, 32, 32]` 的 `img_tensor`。
- 单图 FP32 latency 测量流程：
  - `warmup_runs=50`，先对相同 `img_tensor` 预热 50 次，不计入结果，预热让 CUDA、CuDNN 完成初始化、算法选择和缓存建立；
  - 预热后执行 `torch.cuda.synchronize()`，防止预热任务混进正式计时；
  - 在 `torch.no_grad()` 中用 `time.perf_counter()` 包住 100 次 `model(img_tensor)`；
  - 结束前再次同步 CUDA，避免只测到 CPU 提交异步任务的时间；
  - 100 次总时间除以 `measured_runs=100`，得到 FP32 平均单次 latency；
  - 最后一次 `y_fp32` (输出的形状 `[1, 10]`) 保留下来，用于后续 softmax 和图片标注。
- 单图 TensorRT INT8 latency 使用完全相同的方法：单独预热 50 次、正式测量 100 次、同步 CUDA 并计算平均值，最后一次输出保存在 `y_trt_int8`。
- 两种模型都会输出 10 个类别的 logits。程序用 `softmax` 将其转换为类别概率，并把概率最大的类别及其概率作为 `prediction` 和 `confidence`。

- confidence 只表示模型对当前单图预测结果的置信程度，不是完整测试集 accuracy。
- 程序使用 ImageDraw 把 FP32 和 INT8 的平均 latency、prediction、confidence 写到图片上：
  - 优先加载当前目录的 LiberationSans-Regular.ttf，找不到字体时捕获 OSError，改用 PIL 默认字体；
  - 标注后的图片保存到 exp3/test\_result.jpg，不会覆盖干净输入 test.jpeg。
- 最后调用文件顶部的 test(testmodel)，分别评估 FP32 和 INT8 的完整测试集准确率：
  - with torch.no\_grad() 关闭梯度记录；
  - 遍历 testloader 的 10000 张图片，每个 batch 把 inputs 和 targets 移到 CUDA；
    - outputs.max(1) 取最高 logit 对应的类别；
    - predicted.eq(targets) 判断预测是否正确；
  - correct / total \* 100 得到 accuracy；
  - FP32 与 INT8 测试前后均同步 CUDA，并通过 time.perf\_counter() 统计完整测试集的端到端总耗时。

把这些步骤压缩成一句话就是：run\_exp.sh 激活环境并保存日志，data.py 创建 CIFAR-10 testloader 和 INT8 QDataset，int8\_infer.py 从 exp2 创建 AlexNet、加载 exp3 中的 FP32 权重、用校准集构建 TensorRT INT8 engine，最后比较 FP32/INT8 的单图平均 latency、prediction、confidence 以及完整测试集 accuracy 和总耗时。

## INT8 量化和校准的原理补充

INT8 PTQ (Post-Training Quantization, 训练后量化) 不重新训练 AlexNet，而是在训练完成后把模型转换为适合 INT8 推理的 TensorRT engine。它通常同时涉及 weights 和 activation values。

- **Weights 的量化**：代码中没有单独的 quantize\_weights()。设置 int8\_mode=True 后，TensorRT builder 在构建 engine 时读取模型中已经训练好的 FP32 weights，为支持 INT8 的层计算权重 scale，并在 engine 中保存/使用量化后的权重。
  - Weights 的数值已经固定，因此确定其范围通常不需要校准图片。
- **Activation values 的量化**：activation values 是网络各层在前向传播中产生的输入和输出，会随输入图片变化，因此需要代表性校准图片。int8\_calib\_dataset=cali\_cifar10 让 TensorRT 用 data/q/ 图片执行前向传播并统计各层 activation values 的分布。

ENTROPY\_CALIBRATION\_2 的核心过程可以理解为：

1. 根据校准过程中收集到的 activation values 建立分布/直方图；
2. 尝试不同的截断阈值  $T$ ，把超出  $[-T, T]$  的极端值截断；
3. 对截断后的分布模拟 INT8 量化；
4. 计算量化前后分布的 KLD (Kullback-Leibler Divergence, 相对熵)；
5. 选择使 KLD 最小的  $T$ ，把它作为该张量的动态范围；

6. 再根据  $T$  推导 FP32 与 INT8 之间的 scale。以常见的对称有符号 INT8 为例，可以近似写成：

$$\text{scale} = \frac{T}{127}$$

$$q = \text{clip} \left( \text{round} \left( \frac{x}{\text{scale}} \right), -128, 127 \right)$$

- 其中  $x$  是原始浮点数， $q$  是 INT8 数值。
- 非对称的有符号 INT8 完整范围是  $([-128, 127])$ ；TensorRT 常用的对称 INT8 量化范围是  $([-127, 127])$ ；

所以，正式推理时，weights 和 activation values 会按 TensorRT 为各张量确定的动态范围/scale 使用 INT8 表示。

校准集应该具有代表性，并使用与正式推理相同的 Resize、ToTensor 和 Normalize。如果预处理不同，正式输入产生的 activation values 分布也会改变，校准得到的截断阈值  $T$  和 scale 就可能不再合适，导致更大的量化误差。

## 其它文件/文件夹在实验里的作用

- ./weights/.gitkeep 、 ./weights/alexnet/.gitkeep：让 Git 保留空权重目录。
  - ./weights/alexnet/weights.130.83.050.pt：本实验实际需要的 FP32 AlexNet checkpoint，不提交到 Git。
- ./results\_analysis/run\_exp3\_out.txt：run\_exp.sh 保存的完整 stdout/stderr。
- ../exp1/data/：CIFAR-10 测试集读取/下载位置。
- ../exp2/models/：AlexNet 网络结构来源。
- 一键运行 ./run\_exp.sh 后会生成：

```
exp3/
├─ test_result.jpg
└─ results_analysis/
    └─ run_exp3_out.txt
```

- terminal 和 run\_exp3\_out.txt 会包含：
  - Average Time Spent for fp32: ...ms：FP32 正式推理 100 次的平均单图 latency；
  - Average Time Spent for int8: ...ms：TensorRT INT8 正式推理 100 次的平均单图 latency；
  - Mode: fp32/int8 ...：两种模式对 test.jpeg 的 prediction、confidence 和平均 latency；

- Acc for fp32: ...%; Total time: ...ms : FP32 模型对遍历完整 CIFAR-10 测试集的 accuracy 和 端到端总耗时
  - Acc for trt int8: ...%; Total time: ...ms
- test\_result.jpg 中的两行文字分别对应 FP32 和 INT8:
 

```
Mode: fp32, avg <latency>ms <prediction> (<confidence>%)
Mode: int8, avg <latency>ms <prediction> (<confidence>%)
```
- 注意：单图平均 latency 和完整测试集总耗时是两种不同指标：
  - 单图平均 latency：img\_tensor 已经位于 GPU，只重复执行模型，主要反映模型自身的 GPU 推理时间；
  - 完整测试集总耗时：包含 DataLoader 读取/预处理、Python 循环、CPU 到 GPU 数据复制和模型推理；
  - 因此应当用 FP32 单图平均 latency 对比 INT8 单图平均 latency，用 FP32 完整测试集总耗时对比 INT8 完整测试集总耗时，不能把单图 latency 与整集总时间交叉比较。

## 结果和分析

### 单图推理

| 模式            | 平均 latency | prediction | confidence |
|---------------|------------|------------|------------|
| PyTorch FP32  | 0.482 ms   | bird       | 97.93%     |
| TensorRT INT8 | 0.124 ms   | bird       | 98.43%     |

- 两种模式都把 test.jpeg 预测为 bird，说明本次 INT8 量化没有改变该图片的预测类别。
- INT8 的平均单图 latency 从 0.482 ms 降至 0.124 ms，约为 FP32 的 **3.89 倍加速**，延迟降低约 **74.3%**。
- INT8 confidence 比 FP32 高 0.50 个百分点，但 confidence 只反映模型对这一张图片的置信程度，不能据此判断整个测试集的准确率更高。

### CIFAR-10 测试集

| 模式            | accuracy | 完整测试集总耗时     |
|---------------|----------|--------------|
| PyTorch FP32  | 83.05%   | 14029.809 ms |
| TensorRT INT8 | 82.91%   | 9495.364 ms  |

- INT8 accuracy 比 FP32 低 **0.14 个百分点**，量化造成的精度损失较小。
- 完整测试集总耗时从约 14.03 s 降至 9.50 s，约为 FP32 的 **1.48 倍加速**，总耗时降低约 **32.3%**。
  - 完整测试集的加速比低于单图纯模型推理的加速比，是因为测试集总耗时还包含 DataLoader 读取与预处理、Python 循环、CPU 到 GPU 的数据复制。这些部分不会因模型使用 INT8 而获得同等程度的加速。
- 总体来看，本次 TensorRT INT8 转换以 0.14 个百分点的 accuracy 损失，换取了明显更低的推理延迟和测试集总耗时。